

南开校内 Web 搜索引擎

《信息检索系统原理》HW4 - 课程大作业设计报告

学号：2410668 姓名：吕昊远

1 概述

本次课程大作业面向南开校内资源（*.nankai.edu.cn 等校内域名）实现了一个 Web 搜索引擎。在实现上，系统主要分为四个阶段：**网页抓取**、**文本索引构建**、**检索与排序**以及前端的 **Web 交互**，并在此基础上完成了作业要求的六类高级搜索功能（站内、文档、短语、通配、查询日志、网页快照）与面向登录用户的个性化排序，以及搜索联想与内容推荐两类个性化推荐功能。索引与召回主要基于 Elasticsearch，排序则融合了向量空间模型 (BM25)、链接分析 (PageRank)、时效性与用户偏好多方面因素。接下来将依次说明项目系统的总体设计，并对各模块的具体实现进行详细讲解。

2 系统设计

2.1 处理流程

系统运行分为离线与在线两部分。离线部分主要由爬虫脚本完成抓取，并对抓取得到的网页进行 PageRank 计算、索引构建与联想词典构建；在线部分则使用 Flask 框架结合前端网页提供具体的检索服务。其整体流程如图 1 所示。

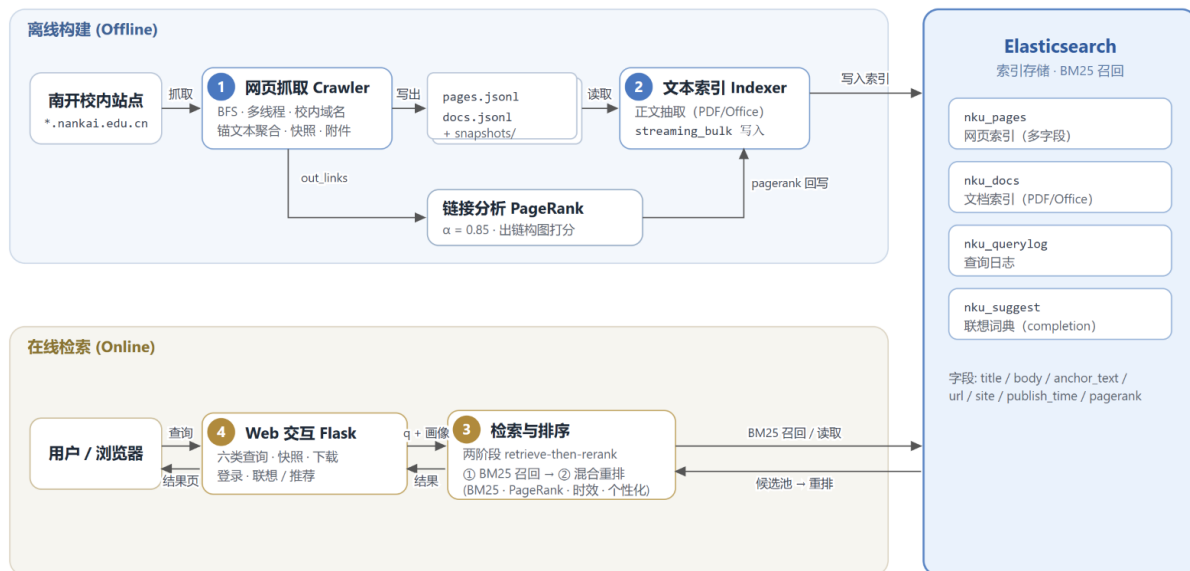


图 1: 系统处理流程

抓取器从种子 URL 出发在校内域名内做广度优先遍历，将网页记录写入 `pages.jsonl`、附件元数据写入 `docs.jsonl`，并抽样对其中部分网页保存快照；PageRank 模块根据 `pages.jsonl` 中的

出链关系结构进行打分；接着索引器读取上述产物，对附件抽取正文后批量写入 Elasticsearch，并将 PageRank 回写到网页文档。

在线检索服务则分为两阶段，先由 ES 用 BM25 召回候选池，再在后端 Python 中按融合公式进行重排；Web 层则负责页面的渲染、网页快照、下载、用户系统与联想推荐等接口。

2.2 技术选型

各模块选型及理由见表 1。

表 1: 技术选型

模块	选型	说明
爬虫	requests + BeautifulSoup + 线程池	校内站点以静态 HTML 为主，多线程即可
索引/检索	Elasticsearch 8.x	自带 BM25、短语/通配、高亮、completion suggester
文档解析	pdfminer.six / python-docx / openpyxl	抽取 PDF/DOCX/XLSX 正文进入文档索引
中文分词	IK	词级切分，召回质量优于单字
链接分析	networkx.pagerank	标准 PageRank, $\alpha = 0.85$
Web 框架	Flask + Jinja2 + Flask-Login	轻量，便于实现用户系统与页面
用户存储	SQLite	单文件、零依赖

在这里我们直接使用了 Elasticsearch 来实现索引与检索的部分工作，使用时只在代码中调用其接口即可实现大部分较为基础的功能。作为一个强大的搜索分析引擎，ES 可以省去很多基础的工作量，从而让我们得以聚焦于高级检索功能的设计逻辑与实现。此外，考虑到网站大部分的检索都为中文，这里直接使用了中文分词器 IK 来进行词级切分。

2.3 模块与目录结构

项目的结构按照职责划分主要有抓取、索引、检索、Web 与脚本五部分，其目录结构如下：

Listing 1: 项目目录结构

```

1 nku_search_engine/
2   config.py           # global config: ES / paths / seeds / weights
3   crawler/ crawler.py # crawling (BFS, threads, anchor text)
4   indexer/ mapping.py # ES index mappings
5                   es_indexer.py # JSONL -> ES, doc text extraction
6   search/ search_service.py # query services + dispatch
7                   ranking.py   # two-stage hybrid re-rank
8                   pagerank.py  # link analysis
9                   recommend.py  # suggest + content recommendation
10  webapp/ app.py        # Flask routes
11         auth.py        # user system (SQLite + Flask-Login)
12         templates/ static/
13  scripts/ run_crawler.py compute_pagerank.py
14         build_index.py build_suggest.py check_data.py
15  data/                                     # runtime: jsonl / pagerank / snapshots / docs / app.db

```

3 模块功能与实现

3.1 网页抓取

抓取器以多线程广度优先遍历校内站点，每条网页记录用 `dataclass` 表示，核心字段有 `url`, `title`, `body`, `anchor_text`, `out_links`, `publish_time`, `snapshot_path`, `site`。对于抓取得到的附件，则是将其另存于 `ext`, `local_path` 等。运行时线程间共享同一线程安全队列与已访问集合，其详细设计如下：

- **域名约束**：仅保留 `host` 以校内后缀结尾的链接 (`is_allowed`)，将抓取范围限定于校内资源。
- **去重**：URL 经 `urldefrag` 去片段后做集合去重；文档 ID 取 `md5(url)`，用作 ES 主键与快照/附件文件名，可以天然防止重复的问题。
- **锚文本聚合**：维护 `anchor_map: url → [anchors]`，在发现链接时累积其他页面对该 URL 的锚文本，待该 URL 被抓取时拼接写入 `anchor_text` 字段，作为高质量的相关性信号单独参与索引的构建。
- **发布时间抽取**：`extract_publish_time` 依次从 `<meta>`、`<time>` 标签与正文前若干字符中匹配日期（支持 2026-03-01、2026 年 3 月 1 日、2026/03/01 等不同格式），将其转为 epoch 秒，以供时效排序计算使用。
- **网页快照**：由于页面数量较多，保存全部原始 HTML 空间占用太大，这里我们采用保存全部关键页面并对其余按照 ID 哈希抽样保存的策略，将少部分快照写入 `snapshots/<doc_id>.html`，其他剩余的则是采用懒保存的方式随检索而新增。
- **附件下载**：识别 `.pdf/.doc/.docx/.xls/.xlsx/.ppt/.pptx` 链接并将附件文件下载到 `docs/`，以供文档查询与下载。
- **抓取规范**：爬虫脚本抓取时，每次请求后按 `CRAWL_DELAY` 休眠以限制其频率，使用标识为学术用途的 `User-Agent`，并以线程数上限约束抓取的并发强度，礼貌进行数据抓取。

具体代码上，抓取主循环对每个 URL 解析其正文与链接，聚合锚文本并将新链接入队，其核心逻辑如下：

Listing 2: 链接抽取、锚文本聚合与 BFS 入队（节选）

```
1 title, body, publish_time, links = self.parse_html(resp, url)
2 for child, anchor in links:
3     if not is_allowed(child):           # keep in-campus links only
4         continue
5     out_links.append(child)
6     with self.lock:
7         if anchor:                       # aggregate anchor text per URL
8             self.anchor_map.setdefault(child, []).append(anchor)
9         if child not in self.visited:    # BFS dedup
10            self.visited.add(child)
11            self.queue.put(child)
12 record = PageRecord(doc_id=url_id(url), url=url, title=title, body=body,
13                    anchor_text=" ".join(self.anchor_map.get(url, [])),
14                    out_links=out_links, publish_time=publish_time, ...)
```

按实验文档要求，抓取页面数为 10 万，完全可以满足常规的检索需求。在系统中也提供了 `scripts/check_data.py` 来统计页面总数、标题/锚文本非空率、含发布时间比例与快照数等指标，方便测试时核验抓取的规模与质量（见第 4 节）。

3.2 文本索引

索引层基于 Elasticsearch，建立了三类主要的索引功能，即网页索引 `nku_pages`、文档索引 `nku_docs` 与查询日志 `nku_querylog`。网页索引设置多个索引域来承担不同检索用途，如表 2。

表 2: 网页索引 `nku_pages` 的主要字段

字段	类型	用途
title	text + keyword(raw)	标题，BM25 高权重；raw 子字段供通配整段匹配
body	text	正文
anchor_text	text	入链锚文本聚合，参与相关性匹配
url / site	keyword	过滤与站内查询限定
publish_time	date(epoch_second)	时效排序
pagerank	float	链接分析分量（归一化至 [0, 1]）

文本字段配置中文 IK 分词器，并为 `title` 建立 `raw` 子字段以支持通配整段匹配。对于附件文档，索引时则按照扩展名调用解析器来抽取正文。PDF 文件使用 `pdfminer.six`，DOCX 按照段落拼接，XLSX 则逐行拼接非空单元格，单篇上限约 20 万字符以控制其体积。最终使用 `streaming_bulk` 进行批量写入，网页索引完成后再将 `pagerank.json` 的分数回写到对应文档。

Listing 3: 网页索引映射（节选）

```
1 PAGE_MAPPING = {"mappings": {"properties": {
2     "title": {"type": "text", "analyzer": ANALYZER,
3         "fields": {"raw": {"type": "keyword"}}},
4     "body": {"type": "text", "analyzer": ANALYZER},
5     "anchor_text": {"type": "text", "analyzer": ANALYZER},
6     "url": {"type": "keyword"},
7     "site": {"type": "keyword"},
8     "publish_time": {"type": "date", "format": "epoch_second"},
9     "pagerank": {"type": "float"}}}}
```

3.3 链接分析 (PageRank)

基于抓取得到的出链关系构造有向图 $G = (V, E)$ （节点为 URL，边为校内超链接），用 `networkx` 计算 PageRank。其迭代定义为：

$$PR(u) = \frac{1 - \alpha}{N} + \alpha \sum_{v \in In(u)} \frac{PR(v)}{L(v)} \tag{1}$$

其中 $N = |V|$ ， $In(u)$ 为指向 u 的页面集合， $L(v)$ 为 v 的出度， $\alpha = 0.85$ 。分数线性归一化到 $[0, 1]$ 后存为 `pagerank.json`，并在建索引时回写到每条网页文档，使其可以参与到重排之中。

Listing 4: PageRank 计算（节选）

```

1 g = nx.DiGraph()
2 for rec in pages:
3     g.add_node(rec["url"])
4     for v in rec.get("out_links", []):
5         g.add_edge(rec["url"], v)
6 pr = nx.pagerank(g, alpha=0.85, max_iter=50, tol=1e-5)
7 mx = max(pr.values()) # normalize to [0,1]
8 pr = {k: v / mx for k, v in pr.items()}

```

3.4 查询服务

查询服务主要实现文档要求的六类高级检索功能，统一入口 `smart_search` 按照查询形态自动进行调度（含 `*`/`?` 则为通配查询，双引号则为短语查询，否则为站内查询），Web 端也可通过 `type` 参数强制指定搜索类型。各功能入口与底层技术见如下表 3 所示。

表 3: 六类查询服务的实现概览

功能	入口	底层实现
站内查询	<code>site_search</code>	多字段 <code>multi_match</code> + <code>site</code> 过滤
文档查询	<code>doc_search</code>	<code>nku_docs</code> 检索 + <code>ext</code> 过滤 + 下载
短语查询	<code>phrase_search</code>	<code>match_phrase</code> （可调 <code>slop</code> ）
通配查询	<code>wildcard_search</code>	<code>wildcard</code> （ <code>*</code> / <code>?</code> ）
查询日志	<code>log_query</code> / <code>/history</code>	写入并回放 <code>nku_querylog</code>
网页快照	<code>/snapshot/<id></code>	渲染抓取时保存的 HTML 快照

接下来将对各项高级检索功能分别进行详细的讲解。

3.4.1 站内查询

对 `title`、`anchor_text`、`body` 做带权 `multi_match`，标题权重最高、锚文本次之，`tie_breaker` 使多字段同时命中者得分更高；指定站点时在外层以 `site` 词项过滤，限定其在某子站内进行检索。

Listing 5: 站内查询

```

1 base = {"multi_match": {"query": q,
2     "fields": ["title^4", "anchor_text^2", "body"],
3     "type": "best_fields", "tie_breaker": 0.3}}
4 if site: # restrict to a sub-site
5     base = {"bool": {"must": [base], "filter": [{"term": {"site": site}]}}}

```

该部分实现较为简单，由于仅抓取校内资源，站内检索的范围已经都在 `nankai` 域名下，对其中学院的域名做区分也很方便，直接对域名进行枚举即可。

3.4.2 文档查询

对 `nku_docs` 按 `title` 与 `content` 检索，支持按文件扩展名（如 `pdf`）过滤；结果对每条命中给出其下载链接 `/download/<id>`，后端按文档 ID 在本地定位文件并以附件形式返回，实现对 PDF/DOCX/XLSX 等附件的查询与获取。

3.4.3 短语查询

用 `match_phrase` 要求词序相邻，对标题/锚文本/正文分别匹配后取并集，并以 `slop` 控制词间间隔。缺省默认 `slop=0` 时，“南开大学”仅在紧邻时命中，而“南开是一所综合性大学”不命中，由此区分“南开大学”与“南开……大学”两类语义。

Listing 6: 短语查询

```
1 {"bool": {"should": [  
2   {"match_phrase": {"title": {"query": q, "slop": 0, "boost": 4}}},  
3   {"match_phrase": {"anchor_text": {"query": q, "slop": 0, "boost": 2}}},  
4   {"match_phrase": {"body": {"query": q, "slop": 0}}}],  
5   "minimum_should_match": 1}}
```

3.4.4 通配查询

支持 `*` (任意多字符) 与 `?` (单字符)。用户模式直接透传给 ES `wildcard`，在 `title.raw` (整段不分词，如“温*”匹配以“温”开头的标题) 与 `title/anchor_text` (词级 token，使“计?”命中“计算”“计网”等两字词) 上并行匹配，并开启 `case_insensitive`；通配限定在短字段以控制开销。

Listing 7: 通配查询

```
1 {"bool": {"should": [  
2   {"wildcard": {"title.raw": {"value": pattern, "boost": 3,  
3     "case_insensitive": true}}},  
4   {"wildcard": {"title": {"value": pattern, "boost": 2,  
5     "case_insensitive": true}}},  
6   {"wildcard": {"anchor_text": {"value": pattern,  
7     "case_insensitive": true}}}],  
8   "minimum_should_match": 1}}
```

3.4.5 查询日志

登录用户每次检索异步写入一条 `{user_id, query, ts}` 到 `nku_querylog`；查询日志所在的 `/history` 页面按照时间倒序记录该用户的历史查询。对于查询日志里的搜索记录，也将作为个性化排序（历史高频词）与搜索联想（用户查询历史）的数据来源。

3.4.6 网页快照

网页快照是搜索引擎对收录网页在抓取时刻的备份，可以有效防止原网页失效后的资源缺失问题。在这部分我们采用了懒备份的方式来生成快照，除了在抓取阶段得到的少部分网页快照外，当用户搜索后访问了某个网页的快照时，如果没有对应的网页快照则会现场对该网页的快照进行抓取与保存。保存的 HTML 存于 `snapshots/`，为隔离快照中的脚本，快照内容经转义后置于 `<iframe srcdoc>` 渲染，并展示存档时间，从而再现抓取时的页面内容。

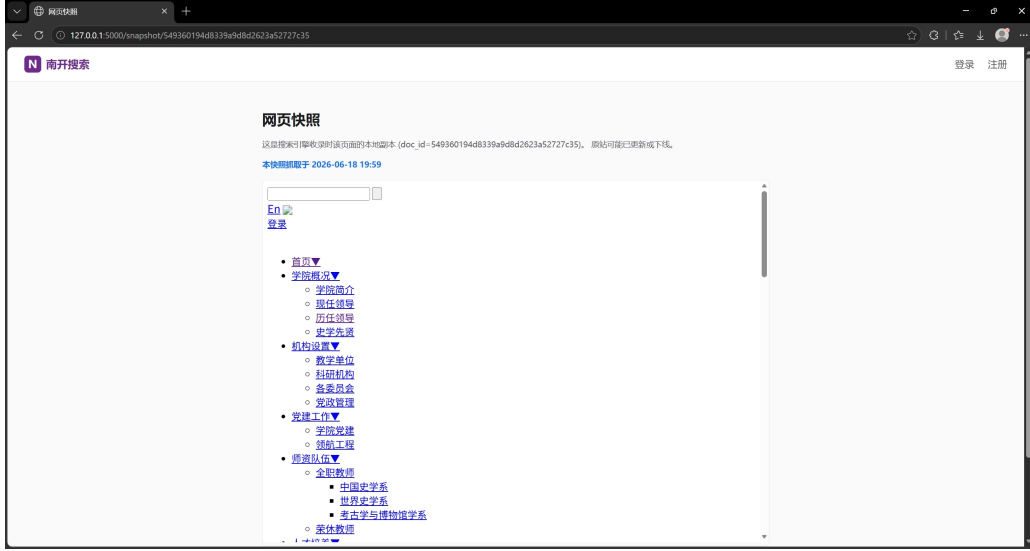


图 2: 网页快照页面

3.5 结果排序

结果排序主要分为两阶段。第一阶段由 ES 以 BM25 召回候选池（默认 100 条）；BM25 在词袋框架下引入词频饱和与文档长度归一化，分数计算公式如下：

$$\text{Score}(Q, d) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)} \quad (2)$$

其中 $k_1 = 1.5$ 、 $b = 0.75$ ， $|d|$ 为文档长度，avgdl 为平均长度。第二阶段则是在 Python 侧将四个归一化分量加权融合，这里登录用户与匿名用户分别使用两套不同权重的计算公式：

$$\begin{aligned} \text{final}_{\text{登录}}(d) &= 0.25 S_{\text{personal}} + 0.10 S_{\text{time}} + 0.10 S_{\text{pagerank}} + 0.55 S_{\text{bm25}} \\ \text{final}_{\text{匿名}}(d) &= 0.10 S_{\text{time}} + 0.15 S_{\text{pagerank}} + 0.75 S_{\text{bm25}} \end{aligned} \quad (3)$$

其中 S_{bm25} 取 ES 原始分除以候选池的最大分， S_{pagerank} 已归一化， S_{time} 、 S_{personal} 详见 3.6 节。此处采用 Python 重排而非纯 ES `function_score`，其原因在于个性化分量依赖用户所在学院关键词与文档内容的匹配，在 Python 侧实现更为直观，且其权重显式给出便于调整，同时也能在结果页展示各分量以核验调参效果。

Listing 8: 候选池融合重排（节选）

```
1 max_bm25 = max((h.get("score") or 0.0) for h in hits) or 1.0
2 for h in hits:
3     bm25 = (h.get("score") or 0.0) / max_bm25          # normalize BM25
4     pr    = float(h.get("pagerank") or 0.0)
5     t_s   = time_score(h.get("publish_time"), role)    # recency
6     p_s   = personal_score(h["title"], h["body"], college, extra_terms)
7     h["final_score"] = (w["personal"]*p_s + w["time"]*t_s
8                        + w["pagerank"]*pr + w["bm25"]*bm25)
9 hits.sort(key=lambda x: x["final_score"], reverse=True)
```

3.6 个性化查询

个性化查询首先基于 SQLite + Flask-Login 的注册/登录系统。注册时填写学院与身份（本科生/研究生/教师），密码经 Werkzeug 哈希存储。在查询上的个性化主要体现在两个方面：

- **学院与关键词词库：** config.COLLEGE_KEYWORDS 为各学院配置主题词集。personal_score 统计这些词在标题（命中记 1.0）与正文（记 0.3）中的出现，历史隐式偏好词各记 0.5，并以饱和和函数归一化：

$$S_{\text{personal}} = \min\left(1, \frac{1}{5} \sum_k w_k\right), \quad w_k \in \{1.0, 0.3, 0.5\}$$
 (4)

- **身份与时效系数：** time_score 按照发布时间给出基础分（表 4），然后乘以身份系数（本科生 ×1.3、研究生 ×1.0、教师 ×0.6），最后再裁剪回 [0, 1] 的归一范围。

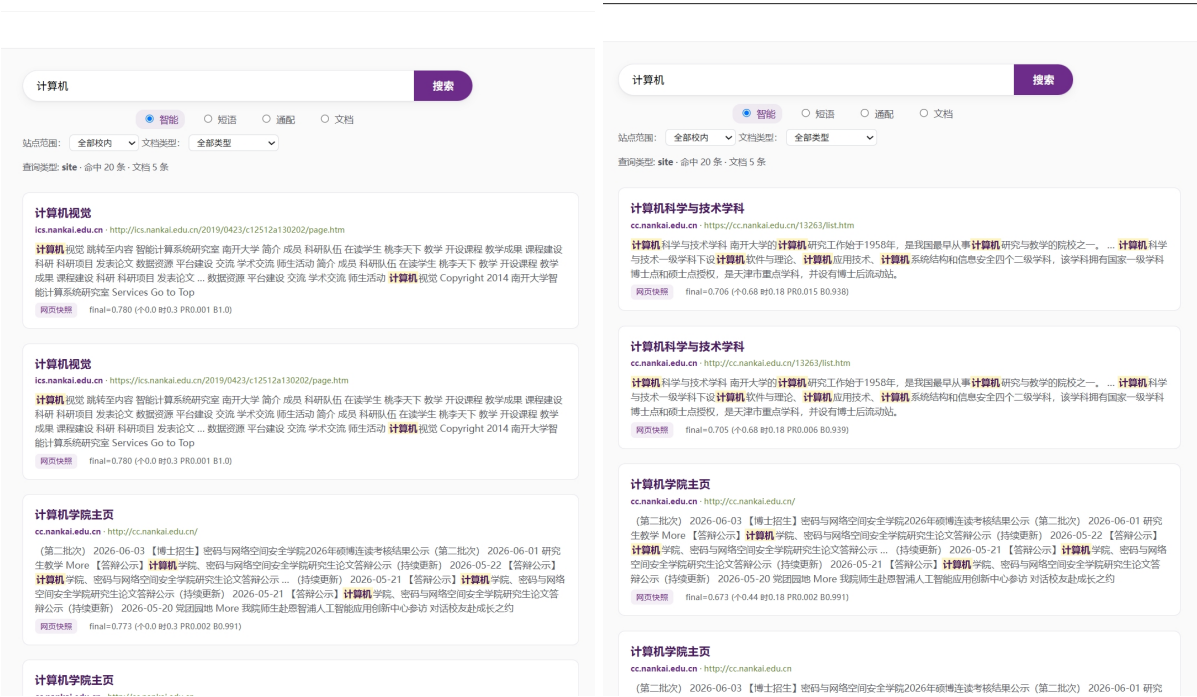


图 3: 同一查询下不同用户的结果排序差异

此外 build_user_profile 用 jieba 切分用户最近查询，取高频词作为隐式偏好 prefer_terms 并入个性化打分。个性化只作用于第二阶段重排、不改变召回，故不会因为偏好而漏掉结果；对同一查询，不同学院/身份的用户只会因为 S_{personal} 与 S_{time} 不同而得到不同的排序（图 3，结果页同时显示 final 分及四个分量）。

表 4: 时效差异（乘身份系数前的基础分）

发布距今	1 天内	1 周内	1 月内	3 月内	1 年内	超 1 年	无日期
基础分	1.0	0.9	0.8	0.6	0.4	0.2	0.3

3.7 个性化推荐

作业要求联想关联与内容推荐二选一，项目对于这两种推荐都做了相关的实现。

搜索联想: 离线由 `build_suggest.py` 对网页标题 jieba 分词, 结合一元词、相邻 bigram/trigram 与查询日志中的热门查询按频次构建词典, 写入 `nku_suggest` 的 `completion` 字段 (底层 FST, 毫秒级前缀匹配)。在线 `suggest` 接口对登录用户先以个人历史中匹配前缀的查询添加到联想推荐词中置顶, 剩余栏位再用全局热词补满; 匿名用户则仅有来自全局热词的联想。具体交互时, 前端搜索框通过监听输入实时进行 `/api/suggest` 的调用。

内容推荐: `recommend_for_user` 取用户的最近查询拼接为文本, 用 ES `more_like_this` 召回相似页面作为首页推荐, 点击卡片即可直接跳转至对应的网址; 匿名或无历史时则回退为按 PageRank 的站内热门页。

Listing 9: 搜索联想: 个人历史置顶 + 全局热词补满

```
1 def suggest(prefix, size=8, user_id=None):
2     results, seen = [], set()
3     if user_id:
4         # personal history first
5         for q in personal_history(user_id, prefix, size):
6             if q not in seen: seen.add(q); results.append(q)
7             if len(results) >= size: return results
8     for w in global_hot(prefix, size):
9         # fill with global hot terms
10        if w not in seen: seen.add(w); results.append(w)
11        if len(results) >= size: break
12    return results
```

3.8 Web 界面

Web 界面基于 Flask + Jinja2, 此处的前端大部分由 AI 完成界面的美化与 UI 设计等。Flask 框架主要路由见表 5; 结果页对每条命中展示标题、站点、命中高亮片段、网页快照链接与排序分量, 搜索联想部分则由搜索框通过 JS 异步实现。

表 5: 主要路由

路由	功能
/	首页: 搜索框 + 内容推荐
/search	结果页: 支持 type=phrase/wildcard/doc 与 site 过滤
/snapshot/<id>	网页快照
/download/<id>	文档下载
/history	查询历史 (登录)
/login /logout /register /profile	用户系统
/api/suggest /api/recommend	联想 / 推荐接口 (JSON)

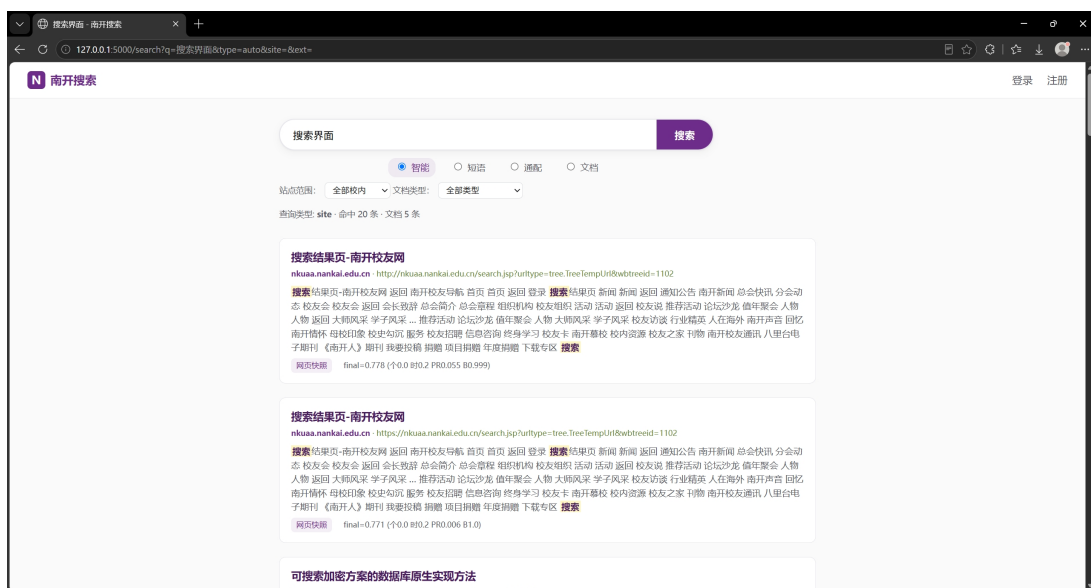
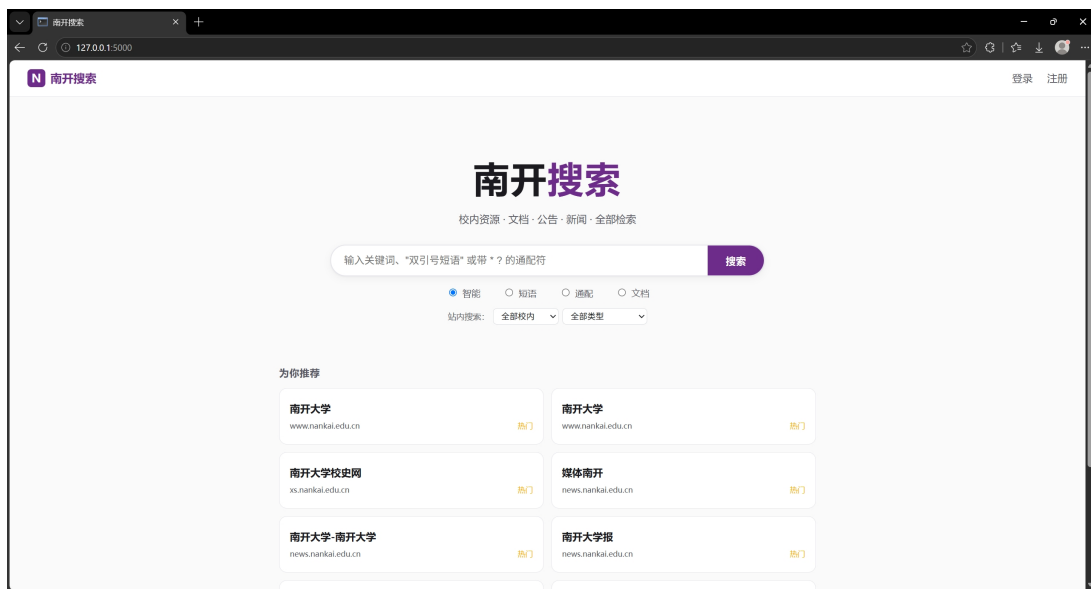


图 4: Web 界面：首页与检索结果页

4 运行与测试

4.1 运行步骤

启动 Elasticsearch 后，依次执行：

Listing 10: 运行流程

```
1 python -m crawler.crawler --max-pages 100000 --workers 16 # 1. crawl
2 python scripts/compute_pagerank.py # 2. PageRank
3 python scripts/build_index.py --recreate # 3. build index
4 python scripts/build_suggest.py --recreate # 4. suggest dict
5 python scripts/run_web.py # 5. start web
```

4.2 数据规模

抓取完成后用 `check_data.py` 自检，关键统计如表 6（按实际抓取结果填写）。

表 6: 抓取数据统计

指标	数值
页面总数 (≥ 10 万)	100,008
标题非空率	99.6%
锚文本非空率	98.8%
含发布时间比例	58.9%
快照文件数	1219
文档条目数	2002

```
=====
NKU 搜索引擎 · 数据自检
=====
页面总数 (pages.jsonl) : 100008
标题非空                : 99618 (99.6%)
锚文本非空              : 98846 (98.8%)
含发布时间              : 58944 (58.9%)
标记了快照              : 1045
快照文件数 (snapshots/) : 1219
文档条目 (docs.jsonl)   : 2002
文档文件数 (docs/)      : 2022
=====
```

图 5: 数据自检脚本输出

4.3 功能测试

对上述各项功能进行测试，结果均符合预期。具体测试过程详见演示视频，此处省略。

5 总结与感悟

本项目依照实验文档要求，完整地实现了南开校内 Web 搜索引擎的各项功能。以 Elasticsearch 为索引与召回基础，采用两阶段 retrieve-then-rerank 排序，将 BM25、PageRank、时效与个性化偏好融合为统一打分；以及六类高级查询功能、基于用户画像的个性化排序、搜索联想与内容推荐的个性化搜索均已实现，并在 Flask 框架下使用前端语言搭建了完整的 Web 交互。

在整体设计上，我们将召回的基础工作交给了成熟组件，任务的重心主要在于 Python 侧的重排与个性化等功能，在工作量与可解释性之间取得了较为合理的平衡。其中 ES 的使用切实减轻了很多工作负担，也让我深刻体会到了这一工具的强大之处，拉近了课程与现实工程的距离。

总之，通过本学期《信息检索系统原理》这门课程的学习，我们已经可以使用成熟的工具来完成一个完整而全面、高效的搜索引擎系统，经历了从原理到实践的完整流程。课上的公式最终转变为了一个可以落地的现实工程系统，既是对于本学期课程学习的检验，也是一次宝贵的实践经历。在未来的改进方向上，也可在抓取层引入更为精细的数据分析，或是在排序层用轻量语义向量来增强个性化的语义理解等，使其更加完善、更加人性化。